

DSA Revision Notes

Topic 9 — Graphs: Shortest Path & Advanced

Dijkstra · Bellman-Ford · Floyd-Warshall · SPFA · Johnson's
MST (Kruskal & Prim) · Bridges & Articulation Points · 2-SAT · 25+ C++ examples

11

Sections

30+

C++ Examples

8

Trace Diagrams

25+

LeetCode Problems

TOC Table of Contents**1. Shortest Path — Overview & When to Use What**

→ Algorithm selection guide · Comparison table

2. Dijkstra's Algorithm — Deep Dive

→ Greedy proof · Lazy deletion heap · Complete C++ implementation
→ Traced step table · Variants: K stops, max probability

3. Bellman-Ford Algorithm

→ Edge relaxation · Negative cycle detection · SPFA optimisation
→ Cheapest Flights Within K Stops

4. Floyd-Warshall — All-Pairs Shortest Path

→ DP formulation · Negative cycle · Path reconstruction
→ Transitive closure · Find city with fewest neighbours

5. Johnson's Algorithm

→ Reweighting with Bellman-Ford · Sparse APSP

6. Minimum Spanning Tree

→ Cut property & cycle property proofs
→ Kruskal's with Union-Find · Prim's with priority queue
→ Min Cost to Connect All Points · Swim in Rising Water

7. Bridges & Articulation Points (Tarjan)

→ Low-link values · DFS tree analysis
→ Find bridges · Find articulation points

8. Euler Path & Circuit

→ Existence conditions · Hierholzer's algorithm
→ Reconstruct Itinerary

9. Network Flow — Max-Flow / Min-Cut

→ Ford-Fulkerson concept · Edmonds-Karp (BFS augmentation)
→ Max-flow = Min-cut theorem

10. 2-SAT (Satisfiability)

→ Implication graph · SCC-based linear solution

11. Complexity Cheat Sheet & LeetCode Map

1 Shortest Path — Overview & Algorithm Selection

1.1 When to Use Which Algorithm

Choosing the right shortest-path algorithm depends on three factors: **graph type** (directed/undirected), **edge weights** (non-negative/negative/0-1), and **query type** (single-source vs all-pairs). Picking the wrong algorithm can mean TLE (Dijkstra on negative edges) or correctness failure (BFS on weighted graphs).

Edge Weights	Single Source	All Pairs	Notes
Unweighted	BFS — $O(V+E)$	BFS $\times V$	Shortest by hop count
Non-negative	Dijkstra $O((V+E)\log V)$	Johnson $O(VE\log V)$	Greedy; no neg edges
0 or 1 only	0-1 BFS — $O(V+E)$	—	Deque trick; faster
Any (neg ok)	Bellman-Ford $O(VE)$	Floyd-Warshall $O(V^3)$	Detect neg cycles
DAG only	DAG Relax $O(V+E)$	—	Topo sort + relax
Dense graph	Dijkstra $O(V^2)$	Floyd-Warshall	Adj matrix Dijkstra

Table 1: Shortest path algorithm selection guide

■ Algorithm Selection Rules

Key invariants to memorise:

BFS → unweighted graphs only (or unit-weight). Shortest path = fewest edges.

Dijkstra → non-negative weights. Fails silently on negative edges — gives WRONG answer.

Bellman-Ford → any weights including negative. Detects negative cycles. $O(VE)$ is slower.

Floyd-Warshall → all-pairs. $O(V^3)$ and $O(V^2)$ space. Only for small V (≤ 500).

0-1 BFS → edge weights in $\{0,1\}$ only. $O(V+E)$. Faster than Dijkstra for this case.

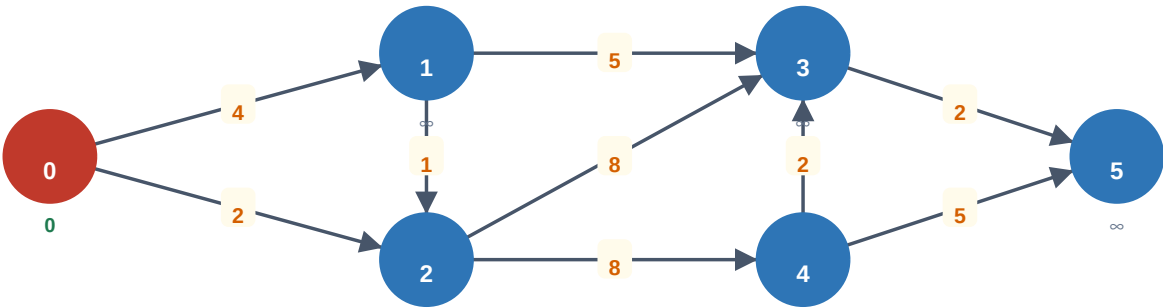
DAG Relax → topological sort then relax edges in order. $O(V+E)$. Handles negative edges.

2

Dijkstra's Algorithm — Deep Dive

2.1 Core Idea & Greedy Proof

Dijkstra maintains a set S of vertices whose shortest path is finalized. At each step, it picks the vertex u NOT in S with the smallest tentative distance, adds it to S, then **relaxes** all edges from u. **Why greedy works:** once dist[u] is selected as minimum, no path through an unvisited vertex can improve it (because all remaining tentative distances are $\geq \text{dist}[u]$ and all edge weights ≥ 0).



Graph for Dijkstra trace: find shortest path from node 0 to all nodes

Step	Visited	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]	dist[5]
Init	{}	0	∞	∞	∞	∞	∞
Pop 0	{0}	0	4	2	∞	∞	∞
Pop 2	{0,2}	0	3	2	10	10	∞
Pop 1	{0,1,2}	0	3	2	8	10	∞
Pop 3	{0..3}	0	3	2	8	10	10
Pop 4	{0..4}	0	3	2	8	10	15
Pop 5	{0..5}	0	3	2	8	10	10

Table 2: Dijkstra step trace (* = current minimum selected for relaxation)

2.2 Full Dijkstra Implementation — Priority Queue

C++

```
1  #include <queue>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5
6  // adj[u] = list of {v, weight}
7  vector<long long> dijkstra(int src, int n,
8                             vector<vector<pair<int,int>>>& adj) {
9      vector<long long> dist(n, LLONG_MAX);
10     // min-heap: {distance, node}
11     priority_queue<pair<long long,int>,
12                   vector<pair<long long,int>>,
13                   greater<pair<long long,int>>> pq;
14
15     dist[src] = 0;
16     pq.push({0LL, src});
17     while (!pq.empty()) {
18         auto [d, u] = pq.top(); pq.pop();
19         // Lazy deletion: skip stale entries
20         if (d > dist[u]) continue;
21         for (auto [v, w] : adj[u]) {
22             if (dist[u] + w < dist[v]) {
23                 dist[v] = dist[u] + w;
24                 pq.push({dist[v], v});
25             }
26         }
27     }
28     return dist; // dist[i] = shortest from src to i, LLONG_MAX if unreachable
29 }
30 // Time: O((V+E) log V)   Space: O(V+E)
```

2.3 Dijkstra with Path Reconstruction

C++

```

1  pair<vector<long long>, vector<int>>
2  dijkstraPath(int src, int n, vector<vector<pair<int,int>>>& adj) {
3      vector<long long> dist(n, LLONG_MAX);
4      vector<int> prev(n, -1);
5      priority_queue<pair<long long,int>,
6                      vector<pair<long long,int>>,
7                      greater<>> pq;
8      dist[src] = 0; pq.push({0, src});
9      while (!pq.empty()) {
10         auto [d, u] = pq.top(); pq.pop();
11         if (d > dist[u]) continue;
12         for (auto [v, w] : adj[u]) {
13             if (dist[u] + w < dist[v]) {
14                 dist[v] = dist[u] + w;
15                 prev[v] = u; // record predecessor
16                 pq.push({dist[v], v});
17             }
18         }
19     }
20     return {dist, prev};
21 }
22
23 vector<int> getPath(int src, int dst, vector<int>& prev) {
24     vector<int> path;
25     for (int v = dst; v != -1; v = prev[v]) path.push_back(v);
26     reverse(path.begin(), path.end());
27     return path[0] == src ? path : vector<int>{}; // empty if unreachable
28 }

```

2.4 LC 743 — Network Delay Time (Medium)

Find time for all nodes to receive a signal from node k. Answer = maximum of all shortest distances from k. If any unreachable, return -1.

C++

```

1  int networkDelayTime(vector<vector<int>>& times, int n, int k) {
2      vector<vector<pair<int,int>>> adj(n+1);
3      for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
4      vector<long long> dist = dijkstra(k, n+1, adj);
5      long long ans = 0;
6      for (int i = 1; i <= n; i++) {
7          if (dist[i] == LLONG_MAX) return -1;
8          ans = max(ans, dist[i]);
9      }
10     return (int)ans;
11 }
12 // Time: O((V+E) log V)   Space: O(V+E)

```

2.5 LC 1514 — Path with Maximum Probability (Medium)

Maximise probability product along a path. Transform: negate $\log(\text{prob})$. Then min-path in transformed graph = max-probability path. Or: adapt Dijkstra to use max-heap, track max-probability instead of min-distance.

```
C++  
  
1  double maxProbability(int n, vector<vector<int>>& edges,  
2      vector<double>& succProb, int start, int end) {  
3      vector<vector<pair<int, double>>> adj(n);  
4      for (int i = 0; i < (int)edges.size(); i++) {  
5          adj[edges[i][0]].push_back({edges[i][1], succProb[i]});  
6          adj[edges[i][1]].push_back({edges[i][0], succProb[i]});  
7      }  
8      vector<double> prob(n, 0.0);  
9      // MAX-heap by probability  
10     priority_queue<pair<double, int>> pq;  
11     prob[start] = 1.0;  
12     pq.push({1.0, start});  
13     while (!pq.empty()) {  
14         auto [p, u] = pq.top(); pq.pop();  
15         if (p < prob[u]) continue;  
16         for (auto [v, w] : adj[u]) {  
17             if (prob[u] * w > prob[v]) {  
18                 prob[v] = prob[u] * w;  
19                 pq.push({prob[v], v});  
20             }  
21         }  
22     }  
23     return prob[end];  
24 }  
25 // Time:  $O((V+E) \log V)$    Space:  $O(V+E)$ 
```

3 Bellman-Ford Algorithm & SPFA

3.1 Bellman-Ford — Algorithm & Proof

Relax ALL edges $V-1$ times. After k iterations, $\text{dist}[v]$ = shortest path using at most k edges. After $V-1$ iterations, shortest paths using at most $V-1$ edges are computed (all simple paths have $\leq V-1$ edges). **Negative cycle detection:** run one more iteration — if any distance still decreases, a negative cycle exists.

```
C++
1 // Edge list: {u, v, weight}
2 vector<long long> bellmanFord(int src, int n,
3                               vector<tuple<int,int,int>>& edges,
4                               bool& hasNegCycle) {
5     vector<long long> dist(n, LLONG_MAX);
6     dist[src] = 0;
7     // Relax all edges V-1 times
8     for (int iter = 0; iter < n - 1; iter++) {
9         bool updated = false;
10        for (auto& [u, v, w] : edges) {
11            if (dist[u] != LLONG_MAX && dist[u] + w < dist[v]) {
12                dist[v] = dist[u] + w;
13                updated = true;
14            }
15        }
16        if (!updated) break; // early termination optimisation
17    }
18    // Check for negative cycles (V-th relaxation still improves)
19    hasNegCycle = false;
20    for (auto& [u, v, w] : edges)
21        if (dist[u] != LLONG_MAX && dist[u] + w < dist[v])
22            hasNegCycle = true;
23    return dist;
24 }
25 // Time: O(V*E)   Space: O(V)
26 // With early termination: O(k*E) where k = actual #iterations needed
```

3.2 LC 787 — Cheapest Flights Within K Stops (Medium)

Exactly $k+1$ edges allowed. Run Bellman-Ford for exactly $k+1$ iterations (not $V-1$). Critical: use a **copy of the previous iteration's dist array** to prevent using newly relaxed edges in the same iteration.

C++

```

1  int findCheapestPrice(int n, vector<vector<int>>& flights,
2                        int src, int dst, int k) {
3      vector<long long> dist(n, LLONG_MAX);
4      dist[src] = 0;
5      // k+1 iterations = at most k stops (k+1 edges)
6      for (int iter = 0; iter <= k; iter++) {
7          vector<long long> prev = dist; // snapshot: avoid using new values
8          for (auto& f : flights) {
9              int u = f[0], v = f[1], w = f[2];
10             if (prev[u] != LLONG_MAX && prev[u] + w < dist[v])
11                 dist[v] = prev[u] + w;
12         }
13     }
14     return dist[dst] == LLONG_MAX ? -1 : (int)dist[dst];
15 }
16 // Time: O(k*E)   Space: O(V)
17 // The prev[] copy is ESSENTIAL: without it you'd allow more than k+1 edges

```

3.3 SPFA — Shortest Path Faster Algorithm

Bellman-Ford optimised with a queue: only re-relax vertices whose distance improved. Average $O(E)$ but worst case still $O(VE)$. Useful when graph is sparse and negative edges exist. Detect negative cycle: if a vertex is enqueued V times, a negative cycle exists.

C++

```

1  vector<long long> spfa(int src, int n,
2                        vector<vector<pair<int,int>>>& adj) {
3      vector<long long> dist(n, LLONG_MAX);
4      vector<bool> inQueue(n, false);
5      vector<int> cnt(n, 0); // enqueue count
6      queue<int> q;
7      dist[src] = 0; inQueue[src] = true; q.push(src);
8      while (!q.empty()) {
9          int u = q.front(); q.pop(); inQueue[u] = false;
10         for (auto [v, w] : adj[u]) {
11             if (dist[u] + w < dist[v]) {
12                 dist[v] = dist[u] + w;
13                 if (!inQueue[v]) {
14                     q.push(v); inQueue[v] = true;
15                     cnt[v]++;
16                     if (cnt[v] >= n) return {}; // negative cycle
17                 }
18             }
19         }
20     }
21     return dist;
22 }
23 // Average: O(E)   Worst: O(VE)   Space: O(V)

```

4 Floyd-Warshall — All-Pairs Shortest Path

4.1 DP Formulation

$dp[k][i][j]$ = shortest path from i to j using only vertices $\{0, 1, \dots, k\}$ as intermediates. Recurrence: $dp[k][i][j] = \min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])$. Space optimised to $O(V^2)$ since we overwrite in-place (same layer).

■ Floyd-Warshall Recurrence

Initialise: $dist[i][i] = 0$ for all i . $dist[i][j]$ = edge_weight(i, j) if edge exists. $dist[i][j] = INF$ otherwise.

For k from 0 to $V-1$: for all i, j : $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$

After V iterations: $dist[i][j]$ = true shortest path between i and j for all pairs.

Negative cycle check: if $dist[i][i] < 0$ for any $i \rightarrow$ negative cycle through i exists.

C++

```

1 // Floyd-Warshall - O(V^3) time, O(V^2) space
2 const long long INF = 1e18;
3
4 vector<vector<long long>> floydWarshall(int n,
5     vector<tuple<int,int,int>>& edges) {
6     // Initialise distance matrix
7     vector<vector<long long>> dist(n, vector<long long>(n, INF));
8     for (int i = 0; i < n; i++) dist[i][i] = 0;
9     for (auto& [u, v, w] : edges) {
10         dist[u][v] = min(dist[u][v], (long long)w);
11         // For undirected: dist[v][u] = min(dist[v][u], (long long)w);
12     }
13     // Relax through each intermediate vertex k
14     for (int k = 0; k < n; k++)
15         for (int i = 0; i < n; i++)
16             for (int j = 0; j < n; j++)
17                 if (dist[i][k] != INF && dist[k][j] != INF)
18                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
19     // Negative cycle: dist[i][i] < 0 for some i
20     return dist;
21 }
22 // Time: O(V^3)   Space: O(V^2)   Use only when V <= 500

```

4.2 Path Reconstruction with Floyd-Warshall

C++

```

1 // next[i][j] = first step on shortest path from i to j
2 vector<vector<int>> next_node;
3
4 void floydWithPath(int n, vector<vector<long long>>& dist) {
5     next_node.assign(n, vector<int>(n, -1));
6     // Initialise: next[i][j] = j if direct edge, -1 if no path
7     for (int i = 0; i < n; i++)
8         for (int j = 0; j < n; j++)
9             if (i != j && dist[i][j] != (long long)1e18)
10                 next_node[i][j] = j;
11     for (int k = 0; k < n; k++)
12         for (int i = 0; i < n; i++)
13             for (int j = 0; j < n; j++)
14                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
15                     dist[i][j] = dist[i][k] + dist[k][j];
16                     next_node[i][j] = next_node[i][k]; // go through k
17                 }
18 }
19
20 vector<int> getPath(int u, int v) {
21     if (next_node[u][v] == -1) return {};
22     vector<int> path = {u};
23     while (u != v) { u = next_node[u][v]; path.push_back(u); }
24     return path;
25 }

```

4.3 LC 1334 — Find the City With Fewest Neighbours (Medium)

C++

```

1 int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
2     vector<vector<long long>> dist(n, vector<long long>(n, 1e9));
3     for (int i = 0; i < n; i++) dist[i][i] = 0;
4     for (auto& e : edges) {
5         dist[e[0]][e[1]] = dist[e[1]][e[0]] = e[2];
6     }
7     // Floyd-Warshall
8     for (int k=0;k<n;k++) for(int i=0;i<n;i++) for(int j=0;j<n;j++)
9         dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j]);
10    // Find city with fewest reachable neighbours within threshold
11    int ans = -1, minNeighbours = n;
12    for (int i = 0; i < n; i++) {
13        int cnt = 0;
14        for (int j = 0; j < n; j++)
15            if (i != j && dist[i][j] <= distanceThreshold) cnt++;
16        if (cnt <= minNeighbours) {
17            minNeighbours = cnt;
18            ans = i; // prefer larger index on tie
19        }
20    }
21    return ans;
22 }
23 // Time: O(V^3)   Space: O(V^2)

```

5 Johnson's Algorithm — Sparse APSP

5.1 Why Johnson's?

Floyd-Warshall is $O(V^3)$ — fine for $V \leq 500$. For larger sparse graphs, running Dijkstra from each vertex is $O(V \cdot (V+E) \cdot \log V)$ — much better when $E \ll V^2$. But Dijkstra doesn't handle negative edges. **Johnson's solution:** reweight all edges to be non-negative using Bellman-Ford potentials, then run Dijkstra from each vertex, then adjust distances back.

■ Johnson's Algorithm Steps

Step 1: Add a new source vertex q with 0-weight edges to all other vertices.

Step 2: Run Bellman-Ford from $q \rightarrow$ get potential $h[v]$ for each vertex.

Step 3: Reweight each edge $(u,v,w) \rightarrow w' = w + h[u] - h[v]$. All $w' \geq 0$.

Step 4: Run Dijkstra from each original vertex using reweighted graph.

Step 5: Recover true distances: $\text{dist}[u][v] = \text{dijkstra_dist}[u][v] - h[u] + h[v]$.

Why $w' \geq 0$? Bellman-Ford ensures $h[v] \leq h[u] + w$ for all edges $\rightarrow w + h[u] - h[v] \geq 0$.

C++

```

1 // Johnson's Algorithm —  $O(V \cdot E + V \cdot (V+E) \cdot \log V)$ 
2 vector<vector<long long>> johnsons(int n,
3     vector<tuple<int,int,int>>& edges) {
4     // Step 1: add virtual source q = n
5     auto ext = edges;
6     for (int i = 0; i < n; i++) ext.push_back({n, i, 0});
7     // Step 2: Bellman-Ford from q
8     vector<long long> h(n+1, 0);
9     for (int iter = 0; iter < n; iter++)
10         for (auto& [u,v,w] : ext)
11             if (h[u] + w < h[v]) h[v] = h[u] + w;
12     // Step 3: reweight — build new adjacency list
13     vector<vector<pair<int,int>>> adj(n);
14     for (auto& [u,v,w] : edges)
15         adj[u].push_back({v, (int)(w + h[u] - h[v])});
16     // Step 4 & 5: run Dijkstra from each vertex
17     vector<vector<long long>> dist(n);
18     for (int src = 0; src < n; src++) {
19         dist[src] = dijkstra(src, n, adj);
20         for (int v = 0; v < n; v++)
21             if (dist[src][v] != LLONG_MAX)
22                 dist[src][v] += h[v] - h[src]; // recover true distance
23     }
24     return dist;
25 }
26 // Time:  $O(V \cdot E + V \cdot (V+E) \cdot \log V)$    Space:  $O(V^2)$ 

```

6 Minimum Spanning Tree — Kruskal & Prim

6.1 MST Properties

A **Minimum Spanning Tree** of a connected weighted undirected graph is a spanning tree with minimum total edge weight. It has exactly $V-1$ edges and contains all vertices. Two foundational properties: **Cut property**: for any cut of the graph, the minimum weight edge crossing the cut is in SOME MST. **Cycle property**: the maximum weight edge in any cycle is NOT in any MST.

6.2 Kruskal's Algorithm — Sort Edges + Union-Find

Sort all edges by weight. Add each edge if it doesn't create a cycle (checked via Union-Find). Stop when $V-1$ edges are added. Greedy proof follows from the cut property.

```
C++
1  struct Edge { int u, v, w; };
2
3  int kruskal(int n, vector<Edge>& edges) {
4      sort(edges.begin(), edges.end(),
5           [](const Edge& a, const Edge& b){ return a.w < b.w; });
6      // Union-Find
7      vector<int> par(n), rnk(n, 0);
8      iota(par.begin(), par.end(), 0);
9      function<int(int)> find = [&](int x) -> int {
10         return par[x] == x ? x : par[x] = find(par[x]);
11     };
12     auto unite = [&](int x, int y) -> bool {
13         x = find(x); y = find(y);
14         if (x == y) return false;
15         if (rnk[x] < rnk[y]) swap(x, y);
16         par[y] = x; if (rnk[x] == rnk[y]) rnk[x]++;
17         return true;
18     };
19     int mstCost = 0, edgesUsed = 0;
20     for (auto& e : edges) {
21         if (unite(e.u, e.v)) {
22             mstCost += e.w;
23             edgesUsed++;
24             if (edgesUsed == n - 1) break;
25         }
26     }
27     return edgesUsed == n - 1 ? mstCost : -1; // -1 if disconnected
28 }
29 // Time: O(E log E)   Space: O(V)
```

6.3 Prim's Algorithm — Priority Queue

Grow the MST one vertex at a time. Start from any vertex, always add the cheapest edge connecting the current MST to a new vertex. Use a min-heap of (weight, vertex). Best for dense graphs (use adjacency matrix $O(V^2)$) or when

starting from a specific vertex.

```
C++
1  int prim(int n, vector<vector<pair<int,int>>>& adj) {
2      vector<bool> inMST(n, false);
3      // min-heap: {edge_weight, vertex}
4      priority_queue<pair<int,int>,
5                      vector<pair<int,int>>,
6                      greater<>> pq;
7      pq.push({0, 0}); // start from vertex 0 with cost 0
8      int mstCost = 0, added = 0;
9      while (!pq.empty() && added < n) {
10         auto [w, u] = pq.top(); pq.pop();
11         if (inMST[u]) continue;
12         inMST[u] = true;
13         mstCost += w;
14         added++;
15         for (auto [v, wt] : adj[u])
16             if (!inMST[v]) pq.push({wt, v});
17     }
18     return added == n ? mstCost : -1;
19 }
20 // Time: O((V+E) log V)   Space: O(V+E)
```

6.4 LC 1584 — Min Cost to Connect All Points (Medium)

Manhattan distance as edge weight. Full graph (all pairs connected). Apply Prim's.

C++

```
1  int minCostConnectPoints(vector<vector<int>>& points) {
2      int n = points.size();
3      // Prim's without adjacency list - O(V^2) for dense graph
4      vector<int> minDist(n, INT_MAX);
5      vector<bool> inMST(n, false);
6      minDist[0] = 0;
7      int total = 0;
8      for (int iter = 0; iter < n; iter++) {
9          // Find vertex with minimum key not in MST
10         int u = -1;
11         for (int i = 0; i < n; i++)
12             if (!inMST[i] && (u == -1 || minDist[i] < minDist[u])) u = i;
13         inMST[u] = true;
14         total += minDist[u];
15         // Update keys of adjacent vertices
16         for (int v = 0; v < n; v++) {
17             if (!inMST[v]) {
18                 int d = abs(points[u][0]-points[v][0])
19                     + abs(points[u][1]-points[v][1]);
20                 minDist[v] = min(minDist[v], d);
21             }
22         }
23     }
24     return total;
25 }
26 // Time: O(V^2)   Space: O(V) - optimal for dense complete graph
```

7 Bridges & Articulation Points — Tarjan's

7.1 Definitions & Core Idea

A **bridge** is an edge whose removal disconnects the graph. An **articulation point** is a vertex whose removal (along with its edges) disconnects the graph. Tarjan's algorithm uses DFS timestamps and **low-link values**: $low[u]$ = minimum discovery time reachable from the subtree rooted at u using at most one back-edge. Edge (u,v) is a bridge if $low[v] > disc[u]$. Vertex u is an articulation point if $low[v] \geq disc[u]$ for some child v .

■ Tarjan's Low-Link Formulas

$disc[u]$ = discovery time of vertex u in DFS

$low[u]$ = min discovery time reachable from subtree of u via one back-edge

Edge (u,v) is BRIDGE: $low[v] > disc[u]$

Vertex u is ARTICULATION POINT: $low[v] \geq disc[u]$ for some child v (AND u is not root, or root has 2+ children)

Low update from tree edge $(u \rightarrow v)$: $low[u] = \min(low[u], low[v])$

Low update from back edge $(u \rightarrow anc)$: $low[u] = \min(low[u], disc[anc])$

C++

```

1  int timer_g = 0;
2
3  // Find all BRIDGES — LC 1192 Critical Connections in a Network
4  void dfsBridge(int u, int parent,
5                vector<vector<int>>& adj,
6                vector<int>& disc, vector<int>& low,
7                vector<pair<int,int>>& bridges) {
8      disc[u] = low[u] = ++timer_g;
9      for (int v : adj[u]) {
10         if (disc[v] == 0) {           // tree edge
11             dfsBridge(v, u, adj, disc, low, bridges);
12             low[u] = min(low[u], low[v]);
13             if (low[v] > disc[u])     // bridge condition
14                 bridges.push_back({u, v});
15         } else if (v != parent) {    // back edge
16             low[u] = min(low[u], disc[v]);
17         }
18     }
19 }
20
21 vector<pair<int,int>> findBridges(int n, vector<vector<int>>& adj) {
22     vector<int> disc(n, 0), low(n, 0);
23     vector<pair<int,int>> bridges;
24     for (int i = 0; i < n; i++)
25         if (!disc[i]) dfsBridge(i, -1, adj, disc, low, bridges);
26     return bridges;
27 }
28 // Time: O(V+E)   Space: O(V)

```

C++

```
1 // Find all ARTICULATION POINTS
2 void dfsAP(int u, int parent,
3           vector<vector<int>>& adj,
4           vector<int>& disc, vector<int>& low,
5           vector<bool>& isAP) {
6     disc[u] = low[u] = ++timer_g;
7     int children = 0;
8     for (int v : adj[u]) {
9         if (!disc[v]) {
10             children++;
11             dfsAP(v, u, adj, disc, low, isAP);
12             low[u] = min(low[u], low[v]);
13             // AP condition 1: non-root with low[v] >= disc[u]
14             if (parent != -1 && low[v] >= disc[u]) isAP[u] = true;
15         } else if (v != parent) {
16             low[u] = min(low[u], disc[v]);
17         }
18     }
19     // AP condition 2: root with 2+ children in DFS tree
20     if (parent == -1 && children > 1) isAP[u] = true;
21 }
22 // Time: O(V+E)   Space: O(V)
```

8 Euler Path & Circuit — Hierholzer's

8.1 Existence Conditions

An **Euler Path** visits every edge exactly once. An **Euler Circuit** is an Euler path that starts and ends at the same vertex.

Graph Type	Euler Circuit Exists	Euler Path Exists
Undirected	All vertices have EVEN degree	Exactly 2 vertices have ODD degree
Directed	Every vertex: in-degree == out-degree	Exactly one vertex: out-deg = in-deg+1 (start), exactly one: in-deg = out-deg+1 (end)
Requirement	Graph must be connected (ignoring isolates)	Graph must be connected

Table 3: Euler path/circuit existence conditions

8.2 Hierholzer's Algorithm — $O(E)$

C++

```
1 // LC 332 - Reconstruct Itinerary (directed Euler path)
2 vector<string> findItinerary(vector<vector<string>>& tickets) {
3     // Build adjacency list (sorted for lexicographic order)
4     map<string, multiset<string>> adj;
5     for (auto& t : tickets) adj[t[0]].insert(t[1]);
6     vector<string> result;
7     stack<string> st;
8     st.push("JFK");
9     while (!st.empty()) {
10         string top = st.top();
11         if (adj[top].empty()) {
12             result.push_back(top); // dead end: add to result
13             st.pop();
14         } else {
15             st.push(*adj[top].begin()); // take smallest neighbour
16             adj[top].erase(adj[top].begin());
17         }
18     }
19     reverse(result.begin(), result.end());
20     return result;
21 }
22 // Time: O(E log E) due to multiset    Space: O(V+E)
23
24 // Generic Euler circuit (undirected, 0-indexed)
25 vector<int> eulerCircuit(int start, vector<vector<int>>& adj) {
26     vector<int> ptr(adj.size(), 0), result;
27     stack<int> st; st.push(start);
28     while (!st.empty()) {
29         int u = st.top();
30         if (ptr[u] == (int)adj[u].size()) {
31             result.push_back(u); st.pop();
32         } else {
33             st.push(adj[u][ptr[u]++]);
34         }
35     }
36     reverse(result.begin(), result.end());
37     return result;
38 }
```

9 Network Flow — Max-Flow / Min-Cut

9.1 Concepts & Max-Flow Min-Cut Theorem

A **flow network** has a source s and sink t , with edges having capacities. The **max-flow** is the maximum amount that can be pushed from s to t . **Residual graph**: for each edge (u,v, cap) , add forward edge (u,v, cap) and backward edge $(v,u,0)$. When we push f units on (u,v) , reduce forward by f and increase backward by f . **Max-Flow = Min-Cut**: the maximum flow equals the capacity of the minimum s - t cut.

■ Network Flow Key Concepts

Augmenting path: any path from s to t in the residual graph with available capacity > 0

Ford-Fulkerson: find augmenting paths repeatedly until none exist. $O(\text{max_flow} * E)$

Edmonds-Karp: use BFS to find shortest augmenting path. $O(V * E^2)$ — polynomial guarantee

Dinic's: BFS level graph + blocking flow. $O(V^2 * E)$. Fastest practical algorithm.

Max-Flow = Min-Cut: proved by Ford-Fulkerson termination — when no augmenting path exists, saturated edges form a minimum cut separating S -reachable from T -reachable vertices.

C++

```

1  // Edmonds-Karp: BFS-based Max-Flow —  $O(V * E^2)$ 
2  struct MaxFlow {
3      int n;
4      vector<vector<int>> cap;    // capacity matrix
5      vector<vector<int>> adj;
6
7      MaxFlow(int n) : n(n), cap(n, vector<int>(n, 0)), adj(n) {}
8
9      void addEdge(int u, int v, int c) {
10         adj[u].push_back(v); adj[v].push_back(u);
11         cap[u][v] += c;        // forward capacity
12         // cap[v][u] stays 0 — backward edge for residual
13     }
14
15     int bfsAugment(int s, int t, vector<int>& parent) {
16         fill(parent.begin(), parent.end(), -1);
17         parent[s] = s;
18         queue<pair<int, int>> q;
19         q.push({s, INT_MAX});
20         while (!q.empty()) {
21             auto [u, flow] = q.front(); q.pop();
22             for (int v : adj[u]) {
23                 if (parent[v] == -1 && cap[u][v] > 0) {
24                     parent[v] = u;
25                     int new_flow = min(flow, cap[u][v]);
26                     if (v == t) return new_flow;
27                     q.push({v, new_flow});
28                 }
29             }
30         }
31         return 0;
32     }
33
34     int maxflow(int s, int t) {
35         int flow = 0;
36         vector<int> parent(n);
37         int new_flow;
38         while ((new_flow = bfsAugment(s, t, parent)) > 0) {
39             flow += new_flow;
40             // Update residual capacities along path
41             int cur = t;
42             while (cur != s) {
43                 int prev = parent[cur];
44                 cap[prev][cur] -= new_flow;
45                 cap[cur][prev] += new_flow;
46                 cur = prev;
47             }
48         }
49         return flow;
50     }
51 };
52 // Time:  $O(V * E^2)$    Space:  $O(V^2)$ 

```

10 2-SAT — Satisfiability via SCC

10.1 2-SAT Problem & Implication Graph

2-SAT asks: given a formula in CNF where every clause has exactly 2 literals, does a satisfying assignment exist? **Key idea:** every clause $(a \text{ OR } b)$ is equivalent to two implications: $(\text{NOT } a \rightarrow b)$ and $(\text{NOT } b \rightarrow a)$. Build an implication graph with $2n$ nodes (one for each variable and its negation). The formula is satisfiable iff no variable x and its negation $\neg x$ are in the same SCC.

■ 2-SAT via SCC

Variable x has two nodes: x (true) and x' (false/negated), indexed as x and $x+n$.

Clause $(a \text{ OR } b)$ adds two directed edges: $a' \rightarrow b$ and $b' \rightarrow a$

Satisfiable: for all i , x_i and x_i' are in DIFFERENT strongly connected components.

Assignment: if $\text{SCC_id}[x_i] > \text{SCC_id}[x_i']$, set $x_i = \text{true}$; else set $x_i = \text{false}$.

(Kosaraju/Tarjan assign SCC IDs in reverse topological order.)

Time: $O(V + E) = O(n + m)$ where $n = \text{\#variables}$, $m = \text{\#clauses}$.

C++

```

1  struct TwoSAT {
2      int n;
3      vector<vector<int>> adj, radj;
4      vector<int> order, comp;
5      vector<bool> vis;
6
7      TwoSAT(int n) : n(n), adj(2*n), radj(2*n), comp(2*n), vis(2*n) {}
8
9      // Add clause: (u OR v). u,v in [0,2n): var i=true → i, var i=false → i+n
10     void addClause(int u, int v) {
11         adj[u^1].push_back(v);    // NOT u → v
12         adj[v^1].push_back(u);    // NOT v → u
13         radj[v].push_back(u^1);
14         radj[u].push_back(v^1);
15     }
16
17     void dfs1(int u) {
18         vis[u] = true;
19         for (int v : adj[u]) if (!vis[v]) dfs1(v);
20         order.push_back(u);
21     }
22     void dfs2(int u, int c) {
23         comp[u] = c;
24         for (int v : radj[u]) if (!comp[v] && comp[v]!=c) dfs2(v,c);
25     }
26     // Returns true if satisfiable, sets assignment
27     bool solve(vector<bool>& assignment) {
28         for (int i = 0; i < 2*n; i++) if (!vis[i]) dfs1(i);
29         int c = 0;
30         fill(comp.begin(), comp.end(), -1);
31         for (int i = 2*n-1; i >= 0; i--)
32             if (comp[order[i]] == -1) dfs2(order[i], c++);
33         assignment.resize(n);
34         for (int i = 0; i < n; i++) {
35             if (comp[i] == comp[i+n]) return false; // x and NOT x in same SCC
36             assignment[i] = comp[i] > comp[i+n];
37         }
38         return true;
39     }
40 };
41 // Time: O(n + m)   Space: O(n + m)

```

11 Complexity Cheat Sheet & LeetCode Map

11.1 Complete Algorithm Complexity Table

Algorithm	Time	Space	Handles Neg?	APSP?	Key Notes
BFS (unweighted)	$O(V+E)$	$O(V)$	No	No	Shortest by hops
0-1 BFS	$O(V+E)$	$O(V)$	No	No	Weights in $\{0,1\}$ only
Dijkstra (heap)	$O((V+E)\log V)$	$O(V)$	No	No	Most common SSSP
Dijkstra (matrix)	$O(V^2)$	$O(V)$	No	No	Better for dense
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Yes	No	Detect neg cycles
SPFA	$O(E)$ avg / $O(VE)$	$O(V)$	Yes	No	Optimised BF
DAG Relax	$O(V+E)$	$O(V)$	Yes	No	DAG only
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Yes	Yes	$V \leq 500$
Johnson's	$O(V \cdot E + V^2 \cdot \log V)$	$O(V^2)$	Yes	Yes	Sparse APSP
Kruskal MST	$O(E \log E)$	$O(V)$	—	—	MST; Union-Find
Prim MST (heap)	$O((V+E)\log V)$	$O(V)$	—	—	MST; dense: $O(V^2)$
Tarjan Bridges/AP	$O(V+E)$	$O(V)$	—	—	Low-link DFS
Hierholzer Euler	$O(E)$	$O(E)$	—	—	Euler path/circuit
Edmonds-Karp Flow	$O(V \cdot E^2)$	$O(V^2)$	—	—	Max-flow BFS
2-SAT	$O(V+E)$	$O(V+E)$	—	—	SCC on impl. graph

Table 4: Complete advanced graph algorithm complexities

11.2 Problem Pattern Recognition

1514	Path with Maximum Probability	Dijkstra (max)	Medium
1584	Min Cost to Connect All Points	Kruskal / Prim MST	Medium
1631	Path With Minimum Effort	Dijkstra binary search	Medium
1697	Checking Existence of Edge Length Limited Paths	Kruskal + offline sort	Hard
1724	Checking Path Length	Bellman-Ford variant	Hard
1786	Number of Restricted Paths	Dijkstra + DP	Medium
2045	Second Minimum Time to Reach Destination	Multi-state BFS	Hard
2203	Minimum Weighted Subgraph With Required Paths	Dijkstra $\times$ 3	Hard
2290	Min Obstacle Removal to Reach Corner	0-1 BFS	Hard
2473	Minimum Cost to Buy Apples	Multi-source Dijkstra	Medium

Table 5: 24 LeetCode problems — Graphs: Shortest Path & Advanced

■ Golden Rules — Graphs: Shortest Path & Advanced

1. Dijkstra: always use lazy-deletion (skip stale pq entries with $d > \text{dist}[u]$). Avoids decrease-key.
2. Bellman-Ford with K stops: use a COPY of dist from previous iteration to prevent chain updates.
3. Floyd-Warshall: loop order must be k (intermediate) outer, i and j inner. Never swap k inward.
4. MST vs Shortest Path: MST minimises total edge weight of spanning tree — NOT shortest between pairs.
5. Kruskal: sort edges $O(E \log E)$, then Union-Find per edge $O(\alpha)$. Total $O(E \log E)$.
6. Bridges condition: $\text{low}[v] > \text{disc}[u]$ (STRICT). Articulation: $\text{low}[v] \geq \text{disc}[u]$ (non-strict).
7. Euler path exists (undirected): exactly 0 or 2 odd-degree vertices. 0 $\rightarrow$ circuit. 2 $\rightarrow$ path.
8. Max-Flow: always add BOTH forward and backward edges in residual graph. Backward starts at 0.
9. 2-SAT: add clause (a OR b) as two edges: NOT-a $\rightarrow$ b and NOT-b $\rightarrow$ a. Then run SCC.
10. 0-1 BFS: weight-0 edges $\rightarrow$ push_front (deque). Weight-1 $\rightarrow$ push_back. $O(V+E)$ — faster than Dijkstra.